工程结构 · 我写的代码

C++

StormfallFollowViewComponent	雨跟随相机	143 行
StormfallWetFootprints	落脚检测 · 脚印/涟漪贴花	188 行
StormfallLightingLibrary	昼夜灯光子关卡切换	76 行

HLSL（材质 Custom 节点）

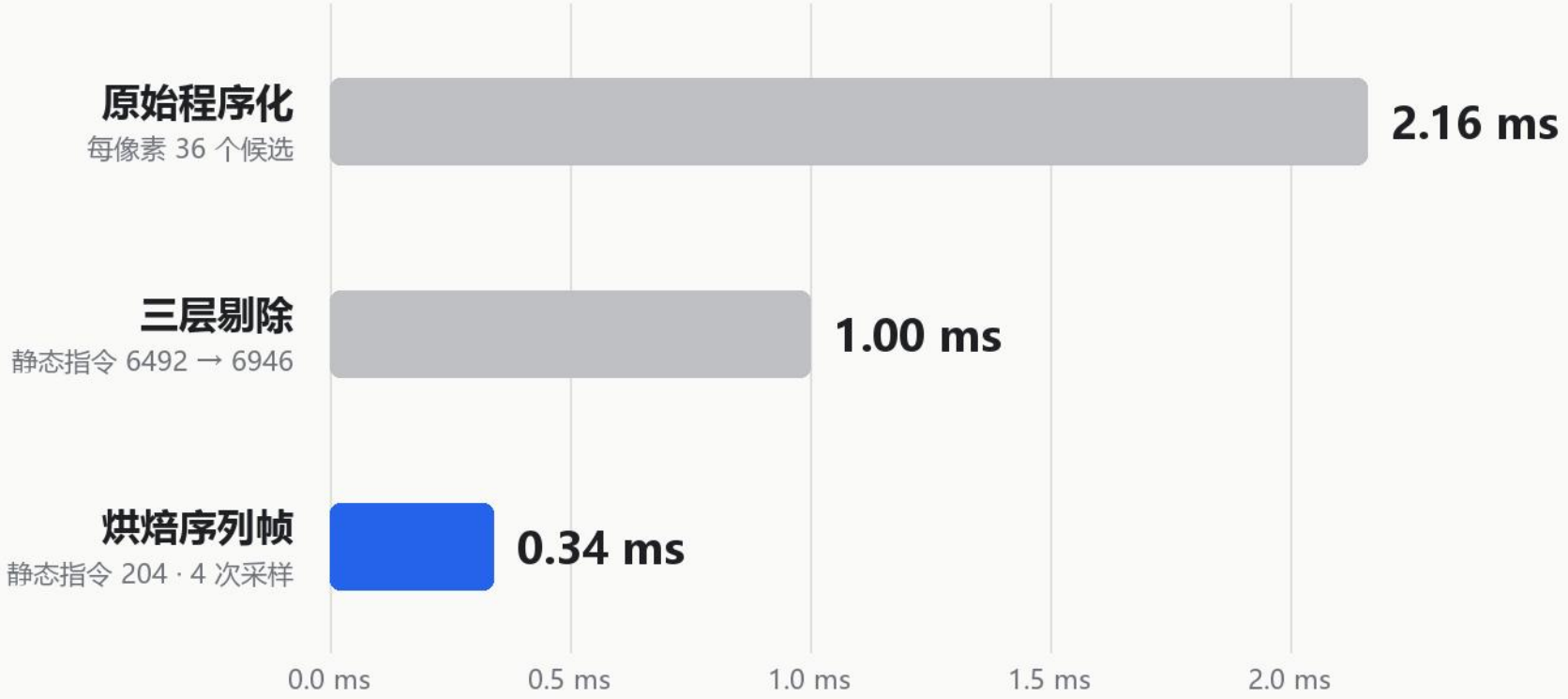
ripple_flipbook.hlsl	烘焙涟漪运行时	47 行
ripple_candidate_v2.hlsl	程序化涟漪（剔除版）	90 行
footprint_decal.hlsl	鞋底脚印 + 变干	32 行
footstep_ripple_decal.hlsl	落脚涟漪（法线贴花）	26 行

Python 工具链

bake_ripple_flipbook.py	离线烘焙无缝序列帧	132 行
ab_ripple.py	ABAB GPU 对比编排	198 行
test_offline.py	离线数值回归	177 行
ue_acceptance.py	自动验收	409 行
buildup_sequence.py	逐层对比展示	114 行

地面涟漪 GPU 开销

R02 · 1920×1080 固定机位 · RTX 4060 Laptop · ABAB 交替实测（开 − 关）



−84%：2.16 → 0.34 ms。程序化版保留为高画质档；烘焙档中远处涟漪弱约 15%。

烘焙涟漪运行时 · HLSL

两层旋转采样 + 帧间混合；未取模坐标求导，跨拼接处 mip 选择正确

```hlsl
// R02 rain ripples, baked tier. Samples T_RippleFlipbook (Tools/R02/bake_ripple_flipbook.py):
// a seamlessly tiling, looping 8x4 atlas of ripple slopes. World centimetres in, tangent normal out.
// Two rotated layers at different scales hide the tile repeat; each layer was baked with half the
// event density, so together they match the procedural version. Adjacent frames are blended.
// Density, size and randomness are fixed at bake time; strength and speed stay live.
// ArtistDensity is only read as an off switch, matching the procedural tier: zero means no ripples.
const float2 kGrid=float2(8,4);              // atlas columns, rows
const float kFrames=32;
const float kTileCm=121.0996;                // one tile = 5 cells of the baked cell size
const float kLoopSeconds=2.6059;             // at the baked ArtistSpeed (downpour preset)
const float kBakedSpeed=0.767253;            // ArtistSpeed used for the bake
const float kSlopeRange=0.5;                 // RG encoding: slope / range * 0.5 + 0.5
const float kSlopeScale=1, kSlopeLimit=.18;
const float kLayerScale=0.731;               // second layer tile size relative to the first
const float2x2 kRotate37=float2x2(0.7986,-0.6018,0.6018,0.7986);
const float2 kFadeCm=float2(2.7,7.6);        // pixel footprint over which ripples fade to flat

float2 p=WorldPos.xy;
float footprint=max(length(ddx(p)),length(ddy(p)));
if(ArtistStrength<=0.0 || ArtistDensity<=0.0 || footprint>=kFadeCm.y)
    return float3(0,0,1);
float distanceFade=1-smoothstep(kFadeCm.x,kFadeCm.y,footprint);

// Unwrapped tile coordinates: their derivatives give correct mips across the frac() seam.
float2 uvA=p/kTileCm;
float2 uvB=mul(kRotate37,p)/(kTileCm*kLayerScale);
float2 dxA=ddx(uvA)/kGrid, dyA=ddy(uvA)/kGrid;
float2 dxB=ddx(uvB)/kGrid, dyB=ddy(uvB)/kGrid;

// Speed slider rescales time; layer B runs half a loop behind so the layers never sync.
float t=Seconds*clamp(ArtistSpeed,0.1,2.0)/kBakedSpeed/kLoopSeconds;
float2 slope=0;
[unroll] for(int layer=0;layer<2;layer++){
 float frameF=frac(t+layer*0.5)*kFrames;
 float f0=floor(frameF), f1=fmod(f0+1,kFrames), w=frameF-f0;
 float2 cell0=float2(fmod(f0,kGrid.x),floor(f0/kGrid.x));
 float2 cell1=float2(fmod(f1,kGrid.x),floor(f1/kGrid.x));
 float2 local=frac(layer==0?uvA:uvB);
 float2 dx=layer==0?dxA:dxB, dy=layer==0?dyA:dyB;
 float2 s0=Flipbook.SampleGrad(FlipbookSampler,(cell0+local)/kGrid,dx,dy).rg;
 float2 s1=Flipbook.SampleGrad(FlipbookSampler,(cell1+local)/kGrid,dx,dy).rg;
 float2 s=(lerp(s0,s1,w)*2-1)*kSlopeRange;
 // The second layer is rotated: rotate its slope back into world axes.
 slope+=layer==0?s:mul(s,kRotate37);
}
slope=clamp(slope*kSlopeScale*clamp(ArtistStrength,0.0,3.0)*distanceFade,-kSlopeLimit,kSlopeLimit);
return normalize(float3(-slope,1.0));
```

Tools/R02/ripple_flipbook.hlsl

湿脚印贴花 · HLSL

鞋底距离场 + 纹路；变干阈值随贴花寿命上升，边缘先干

```hlsl
// Wet footprint decal: shoe-sole print with tread, drying from its edges inwards.
// UV: decal texture coordinates. On a decal oriented by the C++ side, U runs along the foot and
// V across it, so q below is (V,U): q.x across, q.y along. The print is drawn as a left sole
// (big toe towards -q.x); the C++ side mirrors the decal's local Y, which is V, for the right foot.
// Lifetime: DecalLifetimeOpacity, 1 while the print is fresh, falling to 0 over the fade-out.
// Returns opacity of the wet layer (darkening and gloss are applied through it).
const float2 kForeCentre=float2(-0.06,-0.42), kForeRadius=float2(0.90,0.56);  // ball and toes, leaning to the big toe
const float2 kHeelCentre=float2( 0.05, 0.62), kHeelRadius=float2(0.66,0.36);  // heel block
const float2 kArchCentre=float2( 0.22, 0.12), kArchHalf  =float2(0.40,0.34);  // waist, on the outer edge
const float kArchWater=0.55;        // the waist barely touches the ground: a fainter link
const float kTreadPerLength=7.0;    // tread bars along the full sole length (UV -1..1)
const float kTreadDepth=0.65;       // how much less water the grooves leave than the lugs
const float kDryReach=0.9;          // how far into the print the dry front travels by the end

float2 q=(UV*2-1).yx;
// Distance-like field of the sole: 1 inside the pads, 0 on the outline.
float fore=1-length((q-kForeCentre)/kForeRadius);
float heel=1-length((q-kHeelCentre)/kHeelRadius);
float2 a=abs(q-kArchCentre)/kArchHalf;
float arch=(1-max(a.x,a.y))*kArchWater;
float field=max(max(fore,heel),arch);

// Tread: bars across the sole. The lugs carry the water, the grooves between them stay lighter.
float bar=abs(frac(q.y*kTreadPerLength*0.5)-0.5)*2;          // 0 on a lug centre, 1 in a groove
float tread=1-kTreadDepth*smoothstep(0.55,0.85,bar);

// Drying: the threshold rises as the print ages, so the thin rim evaporates first and the
// thickest water, at the pad centres, goes last.
float dry=1-saturate(Lifetime);
float edge=max(EdgeSoftness,0.01);
float wet=smoothstep(dry*kDryReach,dry*kDryReach+edge,field);
return wet*tread*saturate(WetAmount);
```

Tools/WetFootprints/footprint_decal.hlsl

衬衫湿度积分 · C++

每个锚点：淋雨与入水吸水，屋檐下按空气湿度蒸发，再向邻近锚点扩散

```cpp
void UStormfallShirtWetnessComponent::AdvanceWetness(float Seconds)
{
    if(!bInitialized || !FMath::IsFinite(Seconds) || Seconds<=0) return;
    // Bound pathological Blueprint inputs without dropping normal frame time.
    const float Dt=FMath::Min(Seconds,60.f);
    for(int32 I=0;I<WetnessValues.Num();++I)
    {
        const float Rain=Unit(RainAmount)*ExposureValues[I];
        const float Water=ImmersionValues[I];
        const float Absorb=Rain/FMath::Max(.1f,Positive(RainAbsorptionSeconds))+Water/FMath::Max(.1f,Positive(WaterAbsorptionSeconds));
        const float Evaporate=(1-Water)*(1-Rain)*(1-.85f*Unit(AirHumidity))/FMath::Max(.1f,Positive(DryingSeconds));
        WetnessValues[I]=StormfallWetness::Integrate(WetnessValues[I],Absorb,Evaporate,Dt);
    }
    StormfallWetness::Diffuse(WetnessValues,RestPositions,FMath::Clamp(Positive(DiffusionRate),0.f,10.f),Dt,DiffusionRadius);
}
```

Source/Stormfall/StormfallShirtWetness.cpp

泥浆行走痕迹 · Niagara Grid2D HLSL

复用闲置 W 通道；按半衰期衰减并与本次落脚取最大值，不随流体搬运

```
accumulator.xyz=result*opacity+accumulator.xyz*(1-opacity);
accumulator.w=max(accumulator.w,opacity);
strokeIndex++;
}
float4 history;Grid.SamplePreviousGridVector4Value<Attribute=State>(UV+WorldOffsetDelta,history);
if(strokeIndex==0)accumulator=float4(0,0,0,pow(0.95,3));
// Runtime GPU inspection: InputFeedback is .05 after preset initialization.
float3 painted=accumulator.xyz*accumulator.w+history.xyz*(1-accumulator.w);
float4 mesh;Sdf.SamplePreviousGridVector4Value<Attribute=State>(UV,mesh);
painted.xy=mesh.xy*mesh.z+painted.xy*(1-mesh.z);painted.z=max(mesh.z,painted.z);
float edgeValue=saturate(100-0.01-max(abs(UV.x*200-100),abs(UV.y*200-100)));
// Painter W was not consumed by Composite/Divergence/PainterTexture.
// Retain world-space imprints without transporting them with the fluid.
float traceDecay=exp2(-max(DeltaTime,0)/max(TrailHalfLifeSeconds,0.1));
float trace=max(saturate(history.w)*traceDecay,saturate(traceImprint*max(TrailImprintGain,0)));
Value=float4(painted,trace)*(edgeValue>=1?1:0);
#endif
```

Tools/MudStudy/Reference/PainterTraceHistory.hlsl